BTECH (CSE-AIML)

UE24CS341A – SOFTWARE ENGINEERING

PROJECT REPORT

ON

Integrated Capstone idea bank, capstone Project Management, Tracking, and Reporting Platform

SUBMITTED BY

NAME	SRN
TANUSHREE G BHAKTA	PES2UG24AM230
ANUSHA B	PES2UG24AM195
AMEESHA N	PES2UG24AM194
ESHAANA	PES2UG24AM203

TABLE OF CONTENTS

SOFTWARE REQUIREMENTS SPECIFICATION

1. Introduction

1.1 Purpose This document specifies the software requirements for Project 21, the Integrated Capstone Idea Bank, Project Management, Tracking, and Reporting Platform. It outlines the functional and non-functional requirements to guide the development and testing phases.

1.2 Intended Audience and Reading Suggestions The intended audience includes students, faculty guides, system administrators, and the development team.

1.3 Product Scope The platform is a centralized web portal designed to streamline the academic capstone lifecycle. It allows students to browse and vote on capstone ideas, form teams, track milestones, and submit reports. It includes automated workflows for progress reminders and plagiarism checks to assist faculty evaluation.

2. Overall Description

2.1 Product Perspective This system is a new, self-contained web application that replaces manual project tracking. It integrates idea curation, team collaboration, and automated milestone tracking into a single interface.

2.2 Product Functions

- Idea bank management and voting.
- Automated team formation and role assignment workflows.
- Milestone tracking and report submission dashboards.
- Automated progress reminders and document plagiarism checks.

2.3 User Classes and Characteristics

- Students: Will browse ideas, form teams of 3 to 4 members, and submit periodic milestone reports. Every member must contribute to the implementation and complete one full functionality.
- Faculty: Will post ideas, review team formations, evaluate submitted reports, and provide feedback.
- Administrators: Will manage overall system access and moderate the idea bank.

2.4 Operating Environment The platform will operate as a web application accessible via modern browsers. The backend will utilize Node.js, and the application will be containerized using Docker.

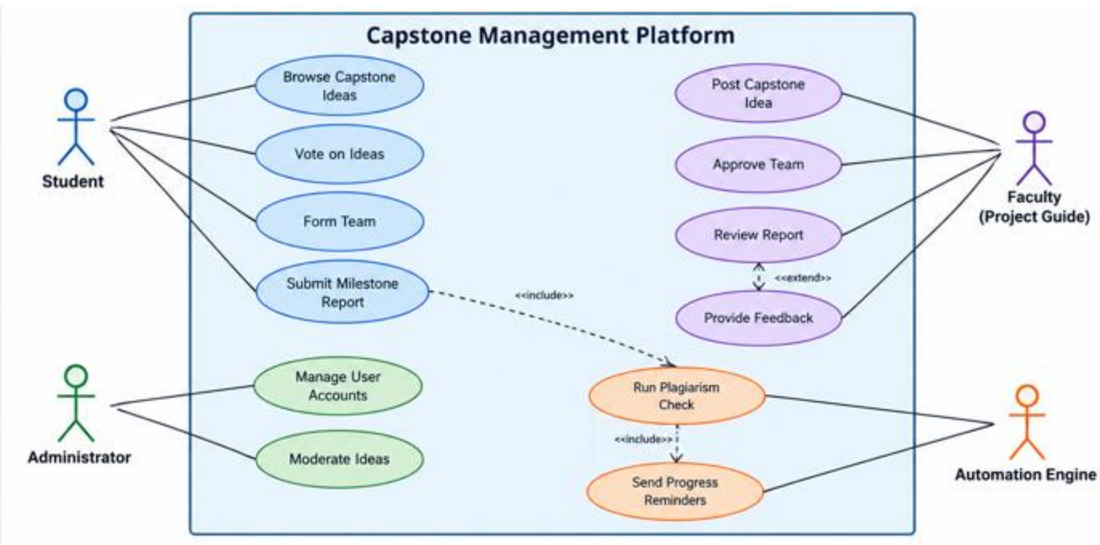
2.5 Design and Implementation Constraints Development must follow Agile methodology, actively using GitHub or Jira for user stories and backlog tracking. Code integration requires a CI/CD pipeline using Jenkins. The backend architecture will utilize a consolidated routing model, strictly omitting a controllers directory, avoiding .cjs files, and integrating authentication directly into route modules rather than utilizing a standalone authController.js file.

3. External Interface Requirements

3.1 User Interfaces The application will feature role-based dashboards with status indicators for project milestones, submission forms for reports, and interactive tables for the idea bank.

3.2 Software Interfaces The system will interface with a distributed version control system (Git/GitHub) for code tracking, Jenkins for Continuous Integration/Continuous Deployment (CI/CD), and external or custom APIs for executing plagiarism checks.

4. Analysis Models



5. System Features

5.1 Idea Bank and Voting

- Description and Priority: High priority. A centralized hub for project topics.
- REQ-1: The system shall allow faculty to post capstone project ideas.
- REQ-2: The system shall allow students to browse, filter, and vote on capstone ideas.

5.2 Team Formation Workflow

- Description and Priority: High priority. Automates group creation.
- REQ-3: The system shall allow students to form and register teams consisting of 3 to 4 members.
- REQ-4: The system shall lock team compositions and assigned roles once faculty approval is granted.

5.3 Milestone Tracking and Reporting

- Description and Priority: High priority. Core tracking mechanism.
- REQ-5: The system shall provide a timeline interface for teams to upload milestone reports.
- REQ-6: The system shall generate automated progress reminders for upcoming milestone deadlines

5.4 Automated Evaluation Assistance

- Description and Priority: Medium priority.
- REQ-7: The system shall automatically execute plagiarism checks on submitted reports and display the results to faculty.

6. Other Nonfunctional Requirements

6.1 Performance Requirements The system must support concurrent access by the entire student batch during submission deadlines with page load times under 2 seconds.

6.2 Security Requirements Security validation is mandatory. The system must implement role-based access control, secure authentication, and data encryption for all academic records.

Appendix C: Requirement Traceability Matrix

Requirement ID	Brief Description	Architecture Reference	Design Reference	Code File Reference	Test Case ID
REQ-1	Faculty post ideas	Arch-UI	UI-Design-1	App.js	UT-01
REQ-2	Students browse & vote	Arch-UI	UI-Design-2	App.js	UT-02
REQ-3	Team formation (3-4 members)	Arch-DB	DB-Schema-1	routes.js	UT-03

Requirement ID	Brief Description	Architecture Reference	Design Reference	Code File Reference	Test Case ID
REQ-4	Lock team compositions	Arch-DB	DB-Schema-1	routes.js	UT-04
REQ-5	Upload milestone reports	Arch-Backend	API-Spec-1	routes.js	UT-05
REQ-6	Progress reminders	Arch-Automation	Cron-Job-1	jobs.js	UT-06
REQ-7	Plagiarism checks	Arch-Backend	API-Spec-2	scanner.js	UT-07